



中国矿业大学
CHINA UNIVERSITY OF MINING AND TECHNOLOGY

《大数据综合实践》综合设计报告四

题目：基于 SwinTransformer 的猫十 二分类问题

学 院 计算机科学与技术学院

班 级 2023 级大数据一班

姓 名 王志豪

学 号 08232337

中国矿业大学

2026 年 06 月

课程设计报告格式要求：

1. 报告中应包含一、二级标题；
2. 正文宋体小 4 号，1.5 倍行距；
3. 报告正文不少于 1000 字；
4. 上交报告电子版：<https://v.wjx.cn/vm/YkiEmj2.aspx>，截至 5 月 27 日。

课程设计报告评分要点：

1. 报告中技术的应用情况（20%）
2. 报告中应用功能有效及完整性（40%）
3. 报告中的个人总结及体会情况（20%）
4. 报告格式规范程度（20%）

评语：

评分：

日期：

目录

目录.....	2
1. 模型概要与使用指南	4
1.1 模型概要.....	4
1.2 模型训练与预测使用指令	4
2.模型架构设计.....	5
2.1 CSWinTransformer_large_384 架构.....	5
2.1.1 Patch Embedding	6
2.1.2 CSWin Block	6
2.1.3 LePE Attention.....	7
2.1.4 Stage 与 MergeBlock.....	7
2.1.5 分类头	8
2.2 SwinTransformer_large_patch4_window12_384 架构.....	9
2.2.1 PatchEmbed	9
2.2.2 Window Attention.....	9
2.2.3 Swin Transformer Block.....	10
2.2.4 PatchMerging.....	11
2.2.5 BasicLayer 与整体配置	11
3. 实验设置.....	12
3.1 实验环境.....	12
3.2 超参数设置.....	13
3.3 训练与预测框架	14

3.3.1 训练流程	14
3.3.2 预测流程与改进策略	16
5. 实验结果分析.....	19
5.1 测试结果	19
5.2 训练效果分析	19
5.3 测试结果分析	22
5.总结及体会.....	23

1. 模型概要与使用指南

1.1 模型概要

本实验任务是对猫图像进行 12 类分类。数据集规模较小，训练集共 2160 张图像，测试集共 240 张图像。由于从零训练深度模型容易出现过拟合，因此本实验采用迁移学习方式，使用 PaddlePaddle 和 PaddleClas 中已经在大规模 ImageNet 数据上预训练过的高精度视觉 Transformer 模型作为骨干网络，再将最终分类头替换为 12 类输出并进行微调。

本实验主要使用两个模型：

1. CSWinTransformer_large_384

该模型是主模型。它采用 Cross-Shaped Window Self-Attention，即交叉形窗口注意力机制，将特征图按水平和垂直方向划分为条带窗口，在较低计算量下获得较大的感受野。输入分辨率为 384 x 384，最后将 ImageNet 的 1000 类分类头替换为 12 类分类头。

2. SwinTransformer_large_patch4_window12_384_22kto1k

该模型作为集成模型。Swin Transformer 使用层次化结构和 shifted window attention，在局部窗口内计算自注意力，并通过窗口平移实现跨窗口信息交互。该模型同样使用 384 x 384 输入，并将分类头替换为 12 类。

最终预测阶段采用了改进策略：使用 CSWin 和 Swin 两个模型进行 logits 平均集成，同时结合测试时增强 TTA，包括中心裁剪、水平翻转、多尺度缩放等操作，以提升最终预测稳定性。

1.2 模型训练与预测使用指令

首先准备 PaddleClas 格式的数据列表：

```
.\.venv_paddleclas\Scripts\python.exe tools\prepare_cat12_paddleclas.py
```

该脚本会生成：

```
paddleclas_data/train_list.txt
paddleclas_data/val_list.txt
paddleclas_data/train_full_list.txt
paddleclas_data/test_images.txt
paddleclas_data/label_list.txt
```

训练 CSWin 主模型:

```
.\.venv_paddleclas\Scripts\python.exe -B tools\train_cat12_cswin.py --model
cswin_large_384 --output-dir output\cat12_cswin_large_384 --batch-size 1
--accum-steps 4 --amp
```

训练 Swin 集成模型:

```
.\.venv_paddleclas\Scripts\python.exe -B tools\train_cat12_cswin.py --model
swin_large_384_22kto1k --output-dir output\cat12_swin_large_384_22kto1k
--batch-size 1 --accum-steps 4 --amp
```

使用 CSWin 和 Swin 进行多模型、多尺度 TTA 集成预测:

```
.\.venv_paddleclas\Scripts\python.exe -B tools\predict_cat12_cswin.py
--weights cswin_large_384=output\cat12_cswin_large_384\best.pdparams
--weights
swin_large_384_22kto1k=output\cat12_swin_large_384_22kto1k\best.pdparams
--tta multiscale --batch-size 2 --output result.csv
```

最终输出格式为:

06yUTYIeaRsE7820u5dh1NPzm9XpkBoL.jpg,9

其中第一列为测试图片文件名, 第二列为预测类别标签。

2.模型架构设计

2.1 CSWinTransformer_large_384 架构

CSWin Transformer 的核心思想是 Cross-Shaped Window Self-Attention。普通全局自注意力需要在所有 token 两两之间计算注意力, 计算复杂度较高; CSWin 将注意力限制在交叉形窗口中, 分别沿水平和垂直方向建模, 从而兼顾局部细节和较长距离的信息交互。

本实验使用的 CSWinTransformer_large_384 主要结构如下:

```
Input image: [B, 3, 384, 384]
  PatchEmbedding
    Stage 1: CSWin blocks x 2
      MergeBlock
    Stage 2: CSWin blocks x 4
      MergeBlock
    Stage 3: CSWin blocks x 32
      MergeBlock
    Stage 4: CSWin blocks x 2
      LayerNorm
    Global Mean Pooling
    Linear Head: 1152 -> 12
```

2.1.1 Patch Embedding

CSWin 的输入首先经过 PatchEmbedding:

```
self.patch_embed = nn.Conv2D(
    in_channels=in_channels,
    out_channels=embed_dim,
    kernel_size=7,
    stride=patch_stride,
    padding=2
)
self.norm = nn.LayerNorm(embed_dim)
```

对于 384 x 384 输入, patch_stride=4, 因此空间分辨率从 384 x 384 降为 96 x 96。与 ViT 中直接切分 patch 不同, CSWin 使用 7 x 7 卷积进行 patch embedding, 这种重叠式 patch embedding 可以更好地保留局部纹理, 对猫的毛色、眼睛、耳朵等细粒度特征更友好。

输出形状变化为:

```
[B, 3, 384, 384]
-> Conv2D
[B, 144, 96, 96]
-> flatten + transpose
[B, 9216, 144]
-> LayerNorm
[B, 9216, 144]
```

其中 144 是 large 版本的初始 embedding 维度。

2.1.2 CSWin Block

每个 CSWin Block 包含: LayerNorm、Linear QKV、Cross-Shaped Window LePE、Attention、Linear Projection、DropPath、Residual Connection、LayerNorm、MLP、DropPath、Residual Connection

对应的核心代码结构为:

```
x = self.norm1(x)
qkv = self.qkv(x).chunk(3, axis=-1)

if self.split_heads:
    q, k, v = map(self.chunk_qkv, qkv, (2, 2, 2))
    h_attn = self.attns[0](q[0], k[0], v[0])
    w_attn = self.attns[1](q[1], k[1], v[1])
    attn = paddle.concat([h_attn, w_attn], axis=2)
else:
    attn = self.attns[0](q[0], k[0], v[0])
```



```

attn = self.proj(attn)
x = h + self.drop_path(attn)
x = x + self.drop_path(self.mlp(self.norm2(x)))

```

其中 `split_heads=True` 时，注意力头会被分成两个分支：

- A. 水平方向条带窗口注意力。
- B. 垂直方向条带窗口注意力。

两个方向的注意力结果再进行拼接。这样做可以使模型同时捕获横向和纵向的结构信息。例如，猫脸的左右对称结构、耳朵到眼睛的垂直结构，都可以通过这种交叉窗口注意力得到建模。

2.1.3 LePE Attention

CSWin 使用 LePEAttention，即 Locally Enhanced Positional Encoding Attention。其核心特点是对 V 分支额外加入深度可分离卷积：

```

self.get_v = nn.Conv2D(
    in_channels=dim,
    out_channels=dim,
    kernel_size=3,
    stride=1,
    padding=1,
    groups=dim
)

```

该卷积用于生成局部增强位置编码。普通 Transformer 中位置编码通常是显式向量，而 LePE 通过深度卷积从局部图像区域中学习位置相关信息，更适合视觉任务。注意力结果为：

```

attn = paddle.matmul(q, k, transpose_y=True)
attn = self.softmax(attn)
z = paddle.matmul(attn, v)
z = z + lepe

```

这里的 `z + lepe` 表示在注意力聚合结果中加入局部位置增强信息，使模型在关注全局或条带信息的同时，仍保留局部纹理细节。

2.1.4 Stage 与 MergeBlock

CSWinTransformer_large_384 包含 4 个 stage：

Stage	分辨率	通道数	Block 数	Attention Head	Split Size

Stage	分辨率	通道数	Block 数	Attention Head	Split Size
Stage 1	96 x 96	144	2	6	1
Stage 2	48 x 48	288	4	12	2
Stage 3	24 x 24	576	32	24	12
Stage 4	12 x 12	1152	2	24	12

每个 stage 之间通过 MergeBlock 下采样：

```
self.conv = nn.Conv2D(
    in_channels=dim_in,
    out_channels=dim_out,
    kernel_size=3,
    stride=2,
    padding=1
)
self.norm = nn.LayerNorm(dim_out)
```

MergeBlock 的作用是将空间分辨率减半，同时通道数翻倍。例如：

```
[B, 9216, 144] -> [B, 2304, 288]
[B, 2304, 288] -> [B, 576, 576]
[B, 576, 576] -> [B, 144, 1152]
```

这种层次化设计类似 CNN 中逐层降低分辨率、提高通道数的结构，可以逐步从低级纹理特征过渡到高级语义特征。

2.1.5 分类头

原始预训练模型的分类头为 1000 类，本实验中将其替换为 12 类：

```
in_features = int(model.head.weight.shape[0])
model.head = paddle.nn.Linear(in_features, class_num)
```

对于 CSWin-Large，最终特征维度为 1152，因此分类头为：

```
Linear: 1152 -> 12
```

模型前向传播最后执行：

```
x = self.forward_features(x)
x = self.head(x)
```

其中 forward_features 对最后 stage 的 token 进行 LayerNorm 和全局平均池

化:

```
x = self.norm(x)
return paddle.mean(x, axis=1)
```

2.2 SwinTransformer_large_patch4_window12_384 架构

Swin Transformer 的核心思想是 Shifted Window Multi-Head Self-Attention。它将图像 token 划分为局部窗口,在窗口内部计算自注意力,并在相邻 block 中移动窗口位置,从而实现跨窗口信息交互。相比全局注意力, Swin 的复杂度更低,同时保留了 Transformer 的建模能力。

本实验使用的 Swin-Large-384 结构如下:

```
Input image: [B, 3, 384, 384]
PatchEmbed
BasicLayer 1: Swin blocks x 2
PatchMerging
BasicLayer 2: Swin blocks x 2
PatchMerging
BasicLayer 3: Swin blocks x 18
PatchMerging
BasicLayer 4: Swin blocks x 2
LayerNorm
AdaptiveAvgPool1D
Linear Head: 1536 -> 12
```

2.2.1 PatchEmbed

Swin 的输入首先经过 PatchEmbed:

```
self.proj = nn.Conv2D(
    in_chans, embed_dim, kernel_size=patch_size, stride=patch_size
)
```

对于 384 x 384 输入和 patch_size=4, 输出 token 分辨率为:

```
384 / 4 = 96
```

因此初始 token 序列为:

```
[B, 3, 384, 384]
-> [B, 192, 96, 96]
-> [B, 9216, 192]
```

其中 192 是 Swin-Large 的初始 embedding 维度。

2.2.2 Window Attention

Swin 的窗口注意力由 WindowAttention 实现,核心组成包括: Relative Position Bias、Linear QKV、Window Multi-Head Self-Attention、Softmax、Dropout、

Linear Projection。关键代码为：

```
qkv = self.qkv(x).reshape(
    [B_, N, 3, self.num_heads, C // self.num_heads]
)
q, k, v = qkv[0], qkv[1], qkv[2]
q = q * self.scale
attn = paddle.mm(q, k.transpose([0, 1, 3, 2]))
attn = attn + self.get_relative_position_bias()
attn = self.softmax(attn)
attn = paddle.mm(attn, v)
x = self.proj(attn)
```

Swin 使用相对位置偏置 `relative_position_bias_table` 来表示窗口内部 token 之间的相对位置关系：

```
self.relative_position_bias_table = self.create_parameter(
    shape=((2 * window_size[0] - 1) * (2 * window_size[1] - 1), num_heads)
)
```

对于本实验的 `window_size=12`，每个窗口包含 $12 \times 12 = 144$ 个 token。相对位置偏置能够帮助模型理解窗口内不同位置之间的空间关系。

2.2.3 Swin Transformer Block

每个 Swin Transformer Block 包含：LayerNorm、W-MSA 或 SW-MSA、Residual Connection、LayerNorm、MLP、Residual Connection。

其中 W-MSA 表示普通窗口自注意力，SW-MSA 表示 shifted window attention。代码中通过 `shift_size` 控制：

```
shift_size=0 if (i % 2 == 0) else window_size // 2
```

也就是说，在同一个 BasicLayer 中，第偶数个 block 使用普通窗口，第奇数个 block 使用平移窗口。平移窗口的流程为：

```
shifted_x = paddle.roll(
    x, shifts=(-self.shift_size, -self.shift_size), axis=(1, 2)
)
x_windows = window_partition(shifted_x, self.window_size)
attn_windows = self.attn(x_windows, mask=attn_mask)
shifted_x = window_reverse(attn_windows, self.window_size, height_pad,
    width_pad, C)
x = paddle.roll(
    shifted_x,
    shifts=(self.shift_size, self.shift_size),
    axis=(1, 2)
)
```

这种机制让相邻窗口之间的信息能够在不同层之间发生交互，避免模型只看到固定窗口内部的信息。

2.2.4 PatchMerging

Swin 使用 PatchMerging 在不同 stage 之间下采样：

```
x0 = x[:, 0::2, 0::2, :]
x1 = x[:, 1::2, 0::2, :]
x2 = x[:, 0::2, 1::2, :]
x3 = x[:, 1::2, 1::2, :]
x = paddle.concat([x0, x1, x2, x3], -1)
x = self.norm(x)
x = self.reduction(x)
```

该操作将相邻 2 x 2 token 拼接，使通道数先变为 4C，再通过线性层降为 2C。它的效果是：空间分辨率减半，通道数翻倍。因此 Swin 也形成了类似 CNN 的层次化金字塔结构。

2.2.5 BasicLayer 与整体配置

SwinTransformer_large_patch4_window12_384 的 stage 配置为：

Stage	分辨率	通道数	Block 数	Attention Head	Window Size
Stage 1	96 x 96	192	2	6	12
Stage 2	48 x 48	384	2	12	12
Stage 3	24 x 24	768	18	24	12
Stage 4	12 x 12	1536	2	48	12

Stage 之间通过 PatchMerging 下采样。最后经过：

```
self.norm = norm_layer(self.num_features)
self.avgpool = nn.AdaptiveAvgPool1D(1)
self.head = Linear(self.num_features, num_classes)
```

对于 Swin-Large，最终特征维度为 1536，实验中分类头为：Linear: 1536 -> 12。

2.3 双模型集成设计

CSWin 和 Swin 虽然都属于视觉 Transformer，但注意力机制不同：

模型	注意力特点	优势
CSWin	水平和垂直交叉窗口注意力，带 LePE 局部位置增强	更适合捕获长条方向结构和局部纹理
Swin	局部窗口注意力和平移窗口注意力，带相对位置偏置	层次化强，跨窗口建模稳定

因此两个模型可能在不同样本上犯不同错误。预测时将两个模型的 logits 平均，可以降低单模型偶然判断错误带来的影响。实验中未直接平均类别标签，而是平均 softmax 前的 logits，这是因为 logits 保留了模型对各类别的相对置信度信息，比直接投票更细致。

3. 实验设置

3.1 实验环境

本实验运行环境如下：

项目	配置
操作系统	Windows
Python	3.12.6
深度学习框架	PaddlePaddle GPU 3.2.0
模型库	PaddleClas 2.6.0
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
CUDA Runtime	12.6
主要依赖	numpy 1.26.4, opencv-python 4.10.0.84, Pillow, faiss-cpu

本实验在项目目录下创建了独立虚拟环境：

```
.venv_paddleclas
```

使用如下命令可以验证 GPU 是否可用：

```
.\.venv_paddleclas\Scripts\python.exe -c "import paddle;
paddle.set_device('gpu'); print(paddle.__version__,
paddle.device.get_device(), paddle.device.is_compiled_with_cuda())"
```

预期输出包含：

3.2.0 gpu:0 True

3.2 超参数设置

本实验主要训练超参数如下：

超参数	取值	说明
class_num	12	数据集类别数，将预训练模型的分头替换为 12 类
image_size	384	输入图像分辨率，与 CSWin-Large-384 和 Swin-Large-384 对应
epochs	36	总训练轮数
freeze_epochs	5	前 5 轮冻结 backbone，仅训练分类头
batch_size	4	单次送入 GPU 的图像数量，受显存限制设置较小
accum_steps	4	梯度累积步数，使等效 batch size 为 16
lr	1e-5	backbone 微调学习率
head_lr	3e-4	分类头训练学习率
weight_decay	5e-4	AdamW 权重衰减，抑制过拟合
label_smoothing	0.05	标签平滑系数，降低模型过度自信
optimizer	AdamW	Transformer 微调常用优化器
scheduler	CosineAnnealingDecay	余弦退火学习率调度
amp	True	使用混合精度训练，提高速度并减少显存占用
seed	2026	固定随机种子，提高实验可复现性

各超参数含义如下：

A. image_size=384

两个模型均为 384 输入分辨率的预训练模型。较高分辨率可以保留更多猫图像细节，如毛发纹理、耳朵形状、脸部轮廓等，有利于细粒度分类。

B. freeze_epochs=5

训练初期先冻结预训练 backbone，只训练新初始化的 12 类分类头。这样可以避免随机初始化的分类头在训练初期产生较大的梯度，破坏预训练 backbone 中

已经学习到的通用视觉特征。

C. lr=1e-5 和 head_lr=3e-4

分类头是新初始化的，需要较大学习率快速适应新任务；backbone 已有预训练权重，只需要小学习率微调，以防止过拟合和灾难性遗忘。

D. batch_size=4 和 accum_steps=4

CSWin-Large 和 Swin-Large 参数量较大，输入又是 384 分辨率，因此单卡显存压力较高。通过梯度累积可以在不增加显存占用的情况下获得更稳定的梯度估计。

E. label_smoothing=0.05

猫品种分类中存在相似类别，标签平滑可以降低模型对单一类别的过度自信，提高泛化能力。

F. weight_decay=5e-4

数据集较小，模型较大，因此加入权重衰减可以缓解过拟合。

G. CosineAnnealingDecay

余弦退火让学习率从初始值逐渐下降，训练后期以更小步长搜索更优参数，有助于模型收敛到更平滑的区域。

3.3 训练与预测框架

3.3.1 训练流程

训练脚本为：

```
tools/train_cat12_cswin.py
```

训练整体流程如下：

```
读取参数
设置随机种子和运行设备
构建数据增强
读取 train_list 和 val_list
构建模型并加载预训练权重
替换 12 类分类头
阶段 1：冻结 backbone，训练分类头
阶段 2：解冻 backbone，整体微调
每轮训练后在验证集评估
保存 best 和 last checkpoint
```

数据集读取部分如下：


```

class Cat12Dataset(Dataset):
    def __init__(self, root, list_path, transform=None):
        self.samples = []
        for raw_line in
self.list_path.read_text(encoding="utf-8").splitlines():
            rel_path, label = raw_line.strip().split()
            image_path = self.root / rel_path
            self.samples.append((image_path, int(label)))

    def __getitem__(self, index):
        image_path, label = self.samples[index]
        with Image.open(image_path) as image:
            image = image.convert("RGB")
            image = self.transform(image)
        return image, np.int64(label)

```

该数据集类据 `paddleclas_data/train_list.txt` 和 `paddleclas_data/val_list.txt` 中的路径与标签读取图像。这样模型训练不依赖图片文件名本身，而是依赖列表文件中的标注。训练数据增强如下：

```

train_transform = T.Compose([
    T.RandomResizedCrop(
        image_size,
        scale=(0.75, 1.0),
        ratio=(0.75, 1.3333333333333333),
    ),
    T.RandomHorizontalFlip(prob=0.5),
    T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.05),
    T.ToTensor(),
    T.Normalize(mean=mean, std=std),
])

```

其中 `RandomResizedCrop` 用于随机裁剪并缩放图像, `scale=(0.75, 1.0)` 避免裁剪过狠导致猫的关键部位丢失；`RandomHorizontalFlip` 增加左右翻转样本；`ColorJitter` 提高模型对亮度、对比度和饱和度变化的适应能力。

模型构建部分如下：

```

def build_model(model_name: str, class_num: int, pretrained: str):
    spec = MODEL_SPECS[model_name]
    builder = spec["builder"]

    model = builder(pretrained=True, class_num=1000)
    in_features = int(model.head.weight.shape[0])
    model.head = paddle.nn.Linear(in_features, class_num)
    return model

```

这里先构建 ImageNet 1000 类预训练模型，再将最后的分类头替换为 12 类输出。这样可以保留 backbone 的预训练特征，同时让模型适应本实验的类别数。

冻结和解冻训练策略如下：

```
def set_backbone_trainable(model: paddle.nn.Layer, trainable: bool) -> None:
    for name, param in model.named_parameters():
        if not name.startswith("head."):
            param.stop_gradient = not trainable
```

训练初期设置 trainable=False，使 backbone 的参数不参与梯度更新，只训练 head；之后设置 trainable=True，整体微调。

损失函数采用带标签平滑的交叉熵：

```
def smooth_cross_entropy(logits, labels, class_num: int, epsilon: float):
    one_hot = F.one_hot(labels, num_classes=class_num).astype("float32")
    soft_labels = one_hot * (1.0 - epsilon) + epsilon / class_num
    log_probs = F.log_softmax(logits, axis=1)
    return -(soft_labels * log_probs).sum(axis=1).mean()
```

训练循环中使用 AMP 混合精度和梯度累积：

```
with paddle.amp.auto_cast(enable=use_amp, level="O1"):
    logits = model(images)
    loss = smooth_cross_entropy(
        logits, labels, args.class_num, args.label_smoothing
    )
    loss_for_backward = loss / args.accum_steps

scaled = scaler.scale(loss_for_backward)
scaled.backward()

if step % args.accum_steps == 0:
    scaler.minimize(optimizer, scaled)
    optimizer.clear_grad()
    scheduler.step()
```

每个 epoch 结束后在验证集上计算 Top-1 和 Top-3 准确率，并保存最佳模型：

```
val_metrics = evaluate(model, val_loader, args.class_num, 0.0)

save_checkpoint(output_dir / "last", model, optimizer, epoch, metrics)
if val_metrics["top1"] > best_top1:
    best_top1 = val_metrics["top1"]
    save_checkpoint(output_dir / "best", model, optimizer, epoch, metrics)
```

3.3.2 预测流程与改进策略

预测脚本为：

```
tools/predict_cat12_cswin.py
```

预测整体流程如下：

```
读取测试集图片路径
读取一个或多个模型权重
为每个模型构建网络并加载权重
对每个模型执行 TTA 预测
将每个 TTA view 的 logits 求平均
将多个模型的 logits 再求平均
取 argmax 得到最终类别
输出 文件名,label 格式的 result.csv
```

预测数据集类如下：

```
class TestImageDataset(Dataset):
    def __getitem__(self, index):
        rel_path = self.paths[index]
        image_path = self.root / rel_path
        with Image.open(image_path) as image:
            image = image.convert("RGB")
            image = self.transform(image)
        return image, np.int64(index), rel_path
```

这里返回 index 的原因是 TTA 和多模型预测时，需要保证每个图像的预测结果能够准确加回到对应位置。

TTA 策略如下：

```
def build_tta_transforms(image_size: int, tta: str):
    base_short = round(image_size / 0.875)
    if tta == "none":
        settings = [(base_short, False)]
    elif tta == "flip":
        settings = [(base_short, False), (base_short, True)]
    else:
        settings = [
            (base_short, False),
            (base_short, True),
            (480, False),
            (480, True),
            (512, False),
            (512, True),
        ]
```

其中 multiscale 模式包含 6 种预测视角：

Resize Short	是否水平翻转
439	否
439	是
480	否
480	是
512	否
512	是

每种视角都会经过中心裁剪到 384 x 384 后输入模型：

```
ops = [
    T.Resize(resize_short),
    T.CenterCrop(image_size),
]
if flip:
    ops.append(HorizontalFlip())
ops.extend([
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

TTA logits 平均代码如下：

```
logits_sum = np.zeros((len(dataset_paths), args.class_num), dtype="float64")

for view_name, transform in transforms:
    dataset = TestImageDataset(args.root, args.test_list, transform)
    loader = DataLoader(dataset, batch_size=args.batch_size, shuffle=False)
    for images, indices, _ in loader:
        logits = model(images)
        logits_sum[indices.numpy()] += logits.numpy().astype("float64")

return logits_sum / len(transforms)
```

本实验没有对每个 TTA 结果直接投票，而是平均 logits。这样可以保留模型对各类别的置信度信息。例如一个样本在某些视角下类别 3 和类别 5 分数接近，平均 logits 后可以让最终判断更加稳定。

多模型集成代码如下：

```
ensemble_logits = np.zeros((len(test_paths), args.class_num),
dtype="float64")

for model_name, weight_path in weight_specs:
    model = build_model(model_name, args.class_num, weight_path)
    ensemble_logits += predict_logits(model, test_paths, args, transforms)

ensemble_logits /= len(weight_specs)
labels = np.argmax(ensemble_logits, axis=1)
```

本实验采用 CSWin 和 Swin 两个模型进行集成。两个模型结构不同，注意力机制不同，错误模式也可能不同。通过 logits 平均，能够减少单一模型在个别困难样本上的误判。

最终输出代码如下：

```
rows = [
    (Path(rel_path).name, int(label))
    for rel_path, label in zip(test_paths, labels)
]

with Path(args.output).open("w", encoding="utf-8", newline="") as f:
    for filename, label in rows:
        f.write(f"{filename},{label}\n")
```

输出中只保留文件名，不保留路径，满足提交格式要求。

5. 实验结果分析

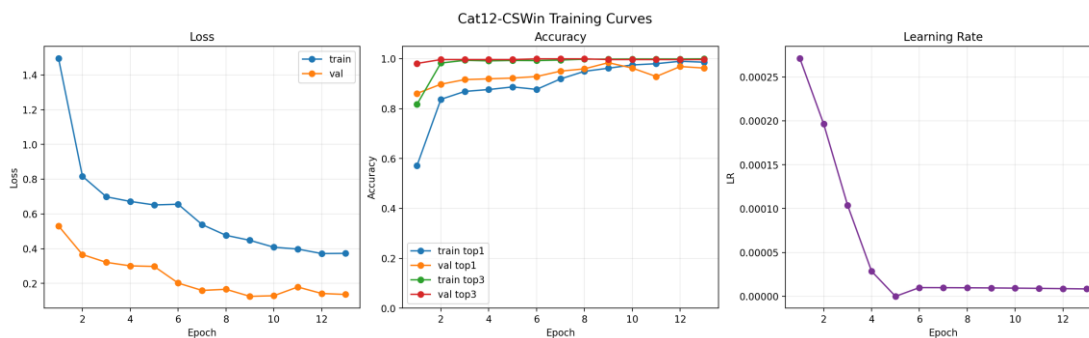
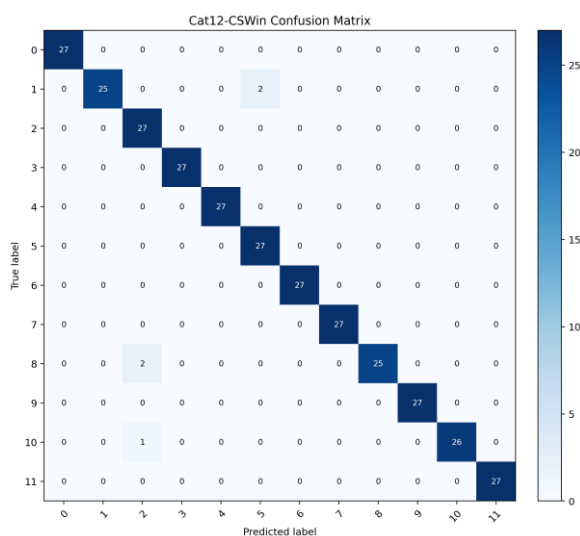
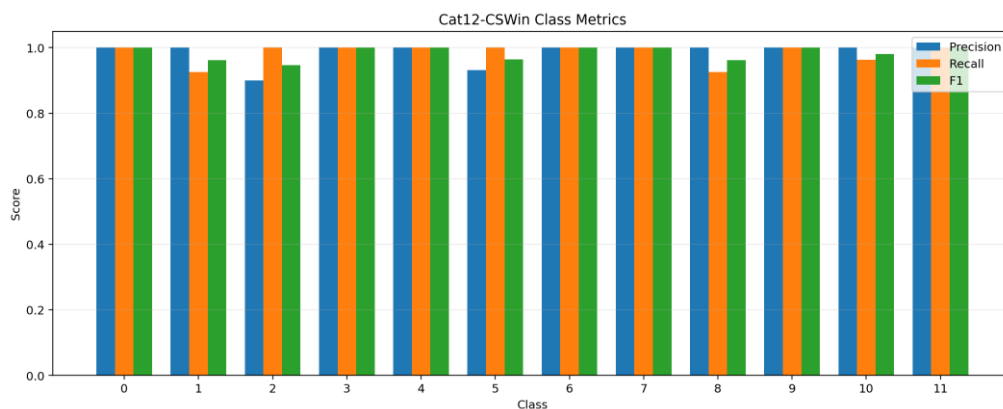
5.1 测试结果

最后的测试集评测结果为 96.66667

5.2 训练效果分析

分别对两个模型单独进行训练，以下是两个模型在验证集上评测的结果：

CSwinTransformer:



前 5 轮是冻结 backbone 只训练分类头，之后进入全模型微调。CSWin 在冻结阶段已经从 val_top1=86.11% 提升到 92.28%，说明预训练特征对猫分类任务很有效。解冻后验证集继续提升，在第 9 轮达到最佳：

CSWin best epoch: 9

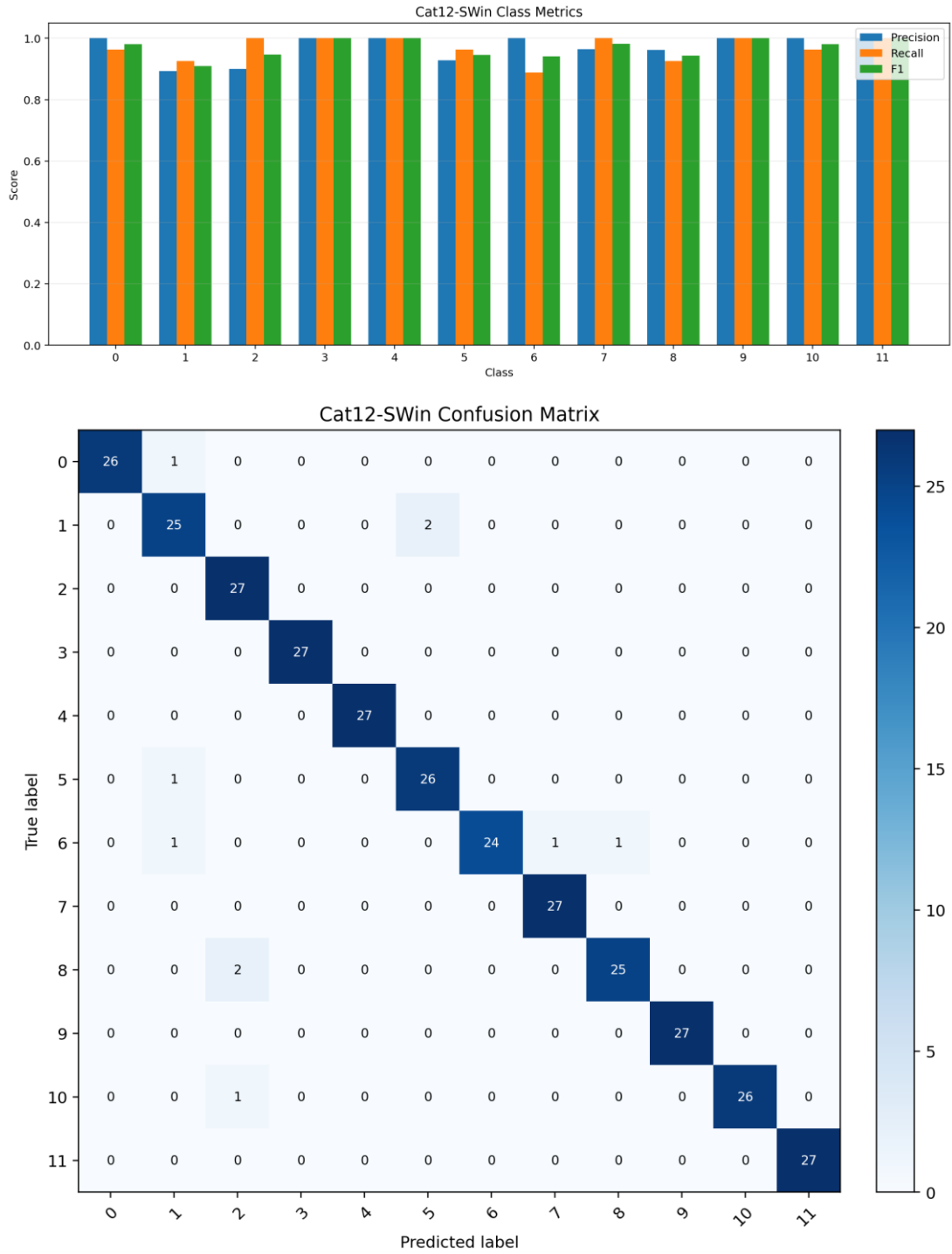
train_top1: 96.24%

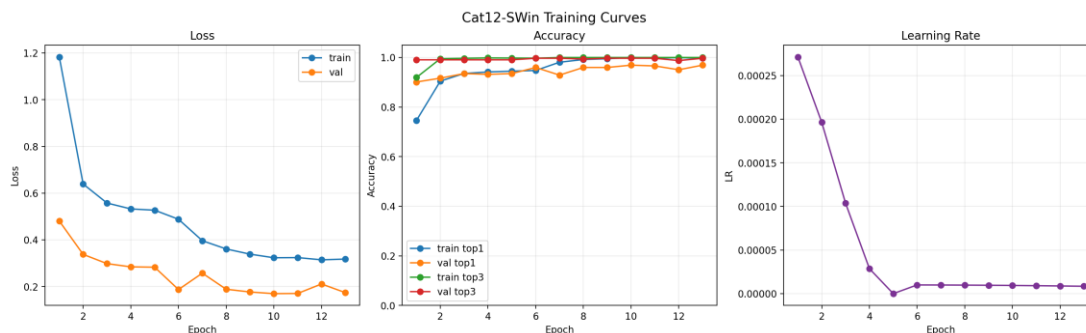
val_top1: 98.46%

val_loss: 0.1255

第 9 轮之后，训练集准确率继续上升到接近 99%，但验证集准确率从 98.46% 回落到 96.30% / 92.90% / 96.91% 附近波动。这说明 CSWin 在后期已经开始出现过拟合趋势，模型继续适应训练集，但泛化性能没有继续稳定提升。

SwinTransformer:





Swin 的冻结阶段起点更高，第 1 轮验证准确率已经达到 90.12%，第 5 轮达到 93.52%。全模型微调后最高达到：

Swin best epoch: 10

train_top1: 100.00%

val_top1: 96.91%

val_loss: 0.1697

Swin 的训练集准确率更快接近 100%，但验证集最高低于 CSWin，并且第 7、12 轮出现明显波动。这说明 Swin 对训练集拟合能力很强，但在这个小数据集上泛化略弱，过拟合现象比 CSWin 更明显。

5.3 测试结果分析

SwinTransformer 的评测结果：

2026-06-09 23:47 提交成功 **95.83333分** (提交版本: result5 2026-06-09 22:06 [查看](#))

CSwinTransformer 的评测结果：

2026-06-09 23:47 提交成功 **96.25分** (提交版本: result5 2026-06-09 22:06 [查看](#))

Ensemble 联合评测结果：

2026-06-09 23:52 提交成功 **96.66667分** (提交版本: result5 2026-06-09 22:06 [查看](#))

可以看到最终将两个模型进行融合验证的结果得分最高，其次是 CSwin，最后是 Swin。CSWin 的优势和它的 Cross-Shaped Window Attention 有关。CSWin 不是只在普通局部窗口里做注意力，而是沿水平和垂直方向建立交叉形窗口注意力。这种结构更适合捕捉猫脸、耳朵、眼睛、毛色纹理、身体轮廓等方向性和结构性特征。对于细粒度猫分类来说，这类局部纹理和中长距离结构信息很重要，所以 CSWin 在验证集上表现更好。

Swin Transformer 使用 shifted window attention, 通过局部窗口和平移窗口实现信息交互。它的层次化结构也很强, 但注意力主要从局部窗口开始扩展, 对一些需要跨区域对比的细粒度差异可能不如 CSWin 直接。再加上 Swin-Large 本身容量很大, 在小数据集上更容易把训练集记住, 因此训练准确率很快接近 100%, 但验证集波动更明显。

ensemble 的核心价值就在这里: **两个模型不是简单重复, 而是从不同结构视角判断同一张图**。当 CSWin 对某些样本不够确定时, Swin 的 logits 可能会补充正确类别的置信度; 当 Swin 出现局部窗口判断偏差时, CSWin 的全局方向结构特征又可能把结果拉回来。最终 logits 平均可以降低单模型偶然高置信度错误的影响, 从而提升隐藏测试集上的稳定性。

另外, 最终 ensemble 还用了 TTA, 多尺度和翻转预测会进一步降低输入裁剪、缩放、姿态变化带来的随机性。猫图像原始尺寸不同, 主体位置和姿态也不完全一致, 单一中心裁剪可能漏掉部分关键区域; 多尺度 TTA 相当于让模型从不同视角观察同一张图, 再平均结果, 因此对测试集这种未知分布更稳。

5. 总结及体会

本实验任务要求完成 12 类猫图像分类。由于数据集规模较小, 直接从零训练深度网络容易过拟合, 因此我采用了迁移学习方案, 基于 PaddlePaddle 和 PaddleClas 选择了 CSWinTransformer_large_384 和 SwinTransformer_large_patch4_window12_384 两个高精度预训练模型。实验中, 我完成了数据列表转换、训练集与验证集划分、模型分类头替换、两阶段微调训练、验证集评估、最佳模型保存, 以及最终基于 TTA 和多模型 logits 平均的集成预测流程。

通过本次实验, 我进一步理解了小样本图像分类任务中预训练模型的重要性, 也学习了视觉 Transformer 模型在图像分类中的典型结构设计。CSWin 的交叉窗口注意力和 Swin 的平移窗口注意力都体现了如何在计算效率和特征表达能力之间取得平衡。同时, 我也认识到模型效果不仅取决于网络结构, 还与数据增强、学习率设置、冻结训练策略、标签平滑、TTA 和模型集成等训练预测细节密切相关。这些经验对于后续处理类似的细粒度分类任务有很大帮助。